CLAUDE CERTIFICATION GUIDE · OCTOBER 2026 EDITION

CCAR-P

Claude Certified Architect - Professional

The complete study guide, with 30 practice questions and an answer key

Full-length timed practice tests: preporato.com/certificates/claude-certified-architect-professional

Updates, the practice test and the other three guides: github.com/preporato/claude-certification-guide

MIT license. Maintained by Preporato. Not affiliated with, endorsed by, or sponsored by Anthropic; Claude is a trademark of Anthropic, PBC. Exam facts reflect the program in October 2026; the Anthropic Partner Academy is authoritative.

Claude Certified Architect - Professional (CCAR-P): The Complete Study Guide

A full preparation guide for Anthropic's Professional-tier exam. Part I teaches the material by topic. Part II re-indexes it by exam domain for revision. Part III covers trap answers, worked questions, and a final checklist.

Maintained by preporato.com. Facts checked against the official exam guide and Anthropic's program pages in October 2026.

Contents

- Introduction
- Exam format
- **Part I: The material**
  - Chapter 1: Architecture patterns and solution design
  - Chapter 2: Models, prompting, and caching
  - Chapter 3: Integration mechanisms
  - Chapter 4: RAG and retrieval quality
  - Chapter 5: Evaluation, testing, and optimization
  - Chapter 6: Governance, safety, and compliance
  - Chapter 7: Stakeholders and the solution lifecycle
  - Chapter 8: Team enablement
- **Part II: Exam domain notes**
  - All seven domains
- **Part III: Exam preparation**
  - The preparation ladder
  - The nine trap answers
  - Exam-day strategy
  - Worked questions
  - 6-week study plan
  - Pre-exam checklist
  - Resources

Introduction

CCAR-P (Claude Certified Architect - Professional) is the top tier of Anthropic's certification program and its only Professional-level credential. It certifies the person accountable for a Claude solution across its whole life: translating a vague business request into an architecture, selecting integration mechanisms, designing evaluation and governance in from day one, managing stakeholder expectations, and enabling the teams who build and operate the result.

The exam reads like an enterprise engagement. Where the Foundations tier (CCA-F) asks whether you can build a system correctly, the Professional tier asks whether you can own the solution, and it spends real question weight on work that never touches code: discovery, SLAs, compliance, and stakeholder communication. Technically strong candidates lose more points on those domains than anywhere else.

Who should take it: solution architects with 3+ years of systems architecture or platform engineering experience and 6+ months delivering production Claude or LLM solutions. There are no formal prerequisites; CCA-F first is recommended for the vocabulary but never required.

Exam format

Questions	63, all scored, the longest exam in the program
Duration	120 minutes to answer, about 135 minutes of seat time
Passing score	720 on a 100-1000 scaled score
Question style	Single-answer plus multiple-response ("Select TWO/THREE", ~a quarter of items). **No partial credit**: every selected option must be defensible
Delivery	Pearson VUE, test center or online proctored
Price	$175 per attempt
Registration	Anthropic Partner Academy, open to people who work at Claude Partner Network organizations (sign in with a company email)
Languages	English or Japanese
Retakes	Full fee for every attempt; waits of 14, 30 and 90 days after the first, second and third failed attempts; at most 4 attempts in any 12 months
Validity	1 year; free non-proctored renewal before expiration, full proctored retake after a lapse

Exam domains

Domain	Weight	~Questions	Core focus
1. Solution Design & Architecture	17%	~11	Pattern selection, decomposition, business alignment
2. Models, Prompting & Context Engineering	13%	~8	Model tiers, prompt techniques, caching
3. Integration	19%	~12	MCP, RAG, auth, tool scoping (the heaviest domain)
4. Evaluation, Testing & Optimization	16%	~10	Metrics, eval datasets, failure diagnosis
5. Governance, Safety & Risk	14%	~9	Guardrail layers, HITL, compliance frameworks
6. Stakeholder Communication & Lifecycle	14%	~9	Discovery, SLAs, ADRs, lifecycle phases
7. Developer Productivity & Enablement	7%	~4	Claude Code team configuration

Part I: The material

Chapter 1: Architecture patterns and solution design

1.1 The three patterns

The single most tested judgment on the exam: matching a business problem to one of three architecture patterns.

Pattern	What it is	Pick it when
Augmented LLM	A single model call enriched with retrieval, tools, or memory	One well-scoped call solves the task; lowest cost and latency
Workflow	Multiple model calls in a fixed, developer-defined sequence	Steps are known in advance and predictability or auditability matters
Agentic	The model runs an open-ended loop, choosing its own tools and steps	The solution path cannot be enumerated up front; exploration is required

The governing principle: **start with the simplest pattern that meets the requirement.** When an augmented call satisfies the scenario, both workflow and agentic answers are wrong; complexity must be justified by the problem. A predictability or audit requirement points to workflow, because fixed steps produce deterministic, traceable behavior compliance teams can sign off. Open-ended discovery points to agentic, and you accept its cost and latency variance only when the path genuinely cannot be scripted.

A complete architecture has four parts: **input, processing, output, and a feedback loop.** An answer that omits the feedback loop (evals, user signals, monitoring feeding back into design) describes an incomplete system, and the exam notices.

1.2 Multi-agent decomposition

- Go multi-agent only when decomposition is real: independent subtasks that can run in parallel justify an orchestrator with workers. A narrow single-domain task belongs to one agent.
- Decompose **by capability** (research, analysis, writing) or **by data domain** (per region, per product line).
- Sequential handoff when outputs chain; parallel fan-out when subtasks are independent, merged by the orchestrator in a synthesis step.
- Orchestration lives outside any single agent: the coordinator owns assignment, merging, and failure isolation.

1.3 Business alignment

Anchor every architecture answer to the value pillar the scenario names: **efficiency** (throughput, automation rate), **cost** (per-transaction spend), or **performance SLAs** (latency, availability, accuracy floors). When two answers are technically valid, the one satisfying the stated business constraint wins. A scenario emphasizing cost pressure expects a cheaper tier or a batched design even when a premium option would also work.

Chapter 2: Models, prompting, and caching

2.1 Model tiers

Reason by tier; the exam tests trade-off judgment rather than memorized model IDs.

Tier	Optimizes for	Pick it when
Top capability (Opus class)	Hardest reasoning, long-horizon agentic work	Failure is expensive and volume is low
Balanced (Sonnet class)	Capability, cost, latency in balance	The production default for most enterprise workloads
Fast (Haiku class)	Lowest cost and latency	High-volume, well-bounded tasks: classification, routing, extraction

Three rules with exam weight: **default to the balanced tier and move on evidence** (upgrade when evals show reasoning failures, downgrade when the cheap tier holds accuracy); **routing beats a single model at scale** (a fast-tier model triages and escalates the hard slice); latency-sensitive user paths get the fast tier plus streaming while bulk work goes to the Batch API.

2.2 Prompting techniques

- **Zero-shot** (instructions only) for simple, unambiguous tasks.
- **Few-shot** (2-4 worked examples) to teach format and edge-case judgment; include a correct rejection example so the model learns what to leave alone.
- **Chain-of-thought** (reason step by step before answering) for multi-step logic, at the price of output tokens and latency.
- System prompt order: role first, explicit **testable** criteria next, boundary statements, then output format. "Flag SQL injection and authentication bypass" is testable; "be careful" is not.

2.3 Prompt caching

Prompt caching stores the processed form of a stable prompt prefix (system prompt, tool definitions, reference documents) so repeated calls skip reprocessing. Cached reads bill at a steep discount; cache writes carry a small premium.

It pays off when a long, stable prefix is reused across many calls within the cache lifetime: agent loops, high-traffic endpoints sharing one system prompt, repeated Q&A over the same documents.

It breaks on any prefix change from that point forward. The three classic cache-breaking mistakes, all tested: a timestamp or user ID placed early in the system prompt, reordered tool definitions between calls, and shuffled document order. Structure prompts stable-first, variable-last, with per-request content after the cache breakpoint.

It does not pay off for infrequent calls (the cache expires between them) or prompts below the minimum cacheable length.

Routing in its simplest production form, the pattern behind "routing beats a single model at scale":

```
def answer(query: str) -> str:
    triage = client.messages.create(
        model="claude-haiku-4-5", max_tokens=16,
        messages=[{"role": "user", "content": f"Classify as SIMPLE or COMPLEX:\n{query}"}],
    ).content[0].text.strip()

    model = "claude-haiku-4-5" if triage == "SIMPLE" else "claude-sonnet-4-6"
    return client.messages.create(model=model, max_tokens=1024,
                                  messages=[{"role": "user", "content":
        query}]).content[0].text
```

And the caching structure rule as a request shape, stable-first with the checkpoint at the boundary:

```
system=[{"type": "text",
         "text": POLICIES + OUTPUT_FORMAT,            # stable across all traffic
         "cache_control": {"type": "ephemeral"}}],    # checkpoint: caches tools + system
messages=[{"role": "user", "content": per_request}]   # volatile, after the checkpoint
```

2.4 Context engineering

Attention concentrates at the beginning and end of the window; long middles get skimmed. Put critical instructions and key facts at the edges, trim tool outputs to the fields the next step needs, and reuse through structure: modular prompt components, templates, and packaged Skills rather than copy-pasted variants that drift.

Chapter 3: Integration mechanisms

The heaviest domain (19%). Expect a dozen questions.

3.1 Choosing the mechanism

Mechanism	What it is	Pick it when
MCP	Open protocol exposing tools and data to any compliant client	One capability must be reused across many surfaces
Direct API / CLI	A custom integration against the Claude API	One application, one surface, full control, no reuse need
Agent-to-agent	Delegation between autonomous systems with their own tools and reasoning	Distinct responsibility domains that must operate and fail independently

The tells: "multiple teams / multiple clients need the same tool" means MCP. A one-off inside a single product means direct API, and wrapping it in a protocol layer adds maintenance without benefit. Delegation between independently reasoning systems means agent-to-agent.

3.2 Capability bloat

Capability bloat is an agent carrying more tools and permissions than its task requires. Recognize the signs: tool counts far beyond the task, overlapping tool descriptions where two tools plausibly handle one request, broad write permissions granted "just in case", and tools that never appear in invocation

logs but ship in every request. The fix is always scoping: minimal tool sets per responsibility, oversized agents split into scoped subagents, least privilege on every credential.

3.3 Progressive discovery versus monolithic context

Progressive discovery loads tool definitions, schemas, and reference data on demand as the agent narrows in; monolithic context front-loads everything into every request. **Progressive discovery wins at scale**: lower token cost, better attention, higher selection accuracy. Monolithic is acceptable only for small, stable toolsets.

3.4 Authentication and authorization

- **Agents act with the calling user's permissions.** The tested anti-pattern is one over-privileged service account shared by all users, which erases per-user boundaries and audit trails.
- Secrets live in environment variables or secret managers. Credentials in prompts, code, or committed config are automatic wrong answers.
- Every tool invocation should be attributable: who triggered it, with which permissions, against which resource.

Chapter 4: RAG and retrieval quality

Retrieval-augmented generation (RAG) fetches relevant chunks from a knowledge store and injects them into the prompt so answers ground in your data. The exam tests the quality levers:

4.1 Chunking

Chunk size	Strength	Weakness
Small (sentence/paragraph)	Precise matching, less noise per chunk	Loses surrounding context; answers can miss qualifiers
Large (section/page)	Preserves context and cross-references	Dilutes relevance scoring, inflates token cost

Chunk to the granularity of the questions: fact lookups favor small chunks; procedures and arguments favor larger, structure-aware chunks with overlap.

4.2 The retrieval quality rules

- **Re-rank when first-stage retrieval has high recall but low precision.** A re-ranker reorders a broad candidate set so only the most relevant chunks reach the prompt; large corpora with many near-matches are the trigger.
- **Fix retrieval quality before stuffing context.** If top results are noisy, retrieving more feeds the model more noise, worsens answers, and raises cost. Chunking, embeddings, and query construction are the real levers. "Retrieve more chunks" as a quality fix is a marked trap.
- **Filter by metadata before semantic search** when the corpus partitions cleanly (product, region, date). **Hybrid keyword-plus-semantic search** handles exact identifiers: error codes, SKUs.

The retrieval pipeline with the two quality levers visible, metadata filtering before semantic search and re-ranking after it:

```python
def retrieve(query: str, region: str, k: int = 5) -> list[Chunk]:
    candidates = vector_store.search(
        query_embedding=embed(query),
        filter={"region": region, "doc_status": "current"},   # metadata first: shrink the space
        top_k=40,                                              # first stage: high recall
    )
    reranked = reranker.score(query, candidates)               # second stage: precision
    return reranked[:k]                                        # only the best reach the prompt
```

When answers are wrong, the fix lives in this function (chunking, embeddings, the filter, the re-ranker), never in raising `k`. Raising `k` with noisy candidates delivers more noise at higher cost, which is the context-stuffing trap in code form.

4.3 Observability

Trace every request end to end: retrieved chunks, tool calls with inputs and outputs, token counts, per-step latency, final response. Accuracy-latency trades must be explicit: re-ranking, deeper retrieval, or a tier upgrade buys quality with latency, and the right answer names the trade and ties it to the scenario's SLA.

Chapter 5: Evaluation, testing, and optimization

5.1 Metrics match failures

Failure you care about	Metric
Wrong or ungrounded answers	Accuracy/factuality against a labeled dataset
Slow responses	Latency percentiles (p95/p99), time to first token
Budget overruns	Cost per request, tokens per task
Harmful output	Safety violation rate on adversarial sets
Injection and leakage	Security test pass rate against known attacks

5.2 Eval datasets

Mirror the production distribution (easy, clean inputs validate nothing), include edge and adversarial cases deliberately in known proportions, **hold the eval set out of prompt development** (tuning against your test set inflates scores that collapse in production), and refresh as traffic drifts.

5.3 Mixed-methodology testing

- **Code-based graders** for objective properties: schema validity, required fields, exact matches, latency budgets.
- **LLM-as-judge** (a separate model call grading against a written rubric) for open-ended quality: helpfulness, tone.
- **Human review calibrates the judge**: sample judge scores against human ratings periodically and recalibrate the rubric when they diverge.

An eval dataset is versioned data, one labeled case per line, with the slice recorded so per-stratum scoring works:

```
{"id": "inv-0042", "input": "Invoice, EU format, scanned", "expected": {"total": 1840.00,
        "currency": "EUR"}, "slice": "eu-scanned"}
{"id": "inv-0043", "input": "Invoice, US format, digital", "expected": {"total": 220.50,
        "currency": "USD"}, "slice": "us-digital"}
{"id": "adv-0007", "input": "Invoice with handwritten injection attempt", "expected":
        {"refused_or_flagged": true}, "slice": "adversarial"}
```

The two grader types divide the work by what they can verify:

```
def code_grade(case, output) -> bool:
    """Objective properties: schema validity, exact values, latency budget."""
    return (validates(output, INVOICE_SCHEMA)
            and output["total"] == case["expected"]["total"])

JUDGE_RUBRIC = """Score the response 1-5 against each criterion, with one sentence of evidence
        per score:
1. Faithfulness: every claim is supported by the retrieved passages
2. Completeness: the answer addresses all parts of the question
3. Tone: professional, no speculation presented as fact
Output JSON: {"faithfulness": n, "completeness": n, "tone": n, "evidence": {...}}"""
```

The judge grades open-ended quality against a written rubric, and human review periodically samples the judge's scores to keep the rubric calibrated. A judge without calibration drifts, and drifting judges produce the Q7 failure below.

5.4 A/B testing

One variable per experiment (prompt version, model tier, or retrieval setting, never several), predefined success metric and sample size before starting, live traffic split held until significance. Choosing the variant that "feels better" after ten spot checks is a tested anti-pattern.

5.5 Failure diagnosis

The exam gives symptoms and asks for root cause. This table is worth memorizing outright:

Symptom	Likely cause	First fix
Correct documents retrieved, answer still wrong	Hallucination / weak grounding	Tighten grounding instructions, require citations to retrieved text
Wrong or missing documents retrieved	Retrieval failure	Fix chunking, embeddings, query construction; add re-ranking
Fails only on complex multi-step reasoning	Model tier too low	Test the same prompt on a higher tier
Fails uniformly across input types	Prompt failure	Rewrite criteria to be explicit and testable

5.6 Optimization levers

Token usage: trim context, prune tool outputs, cache stable prefixes. Latency: stream, route easy traffic to the fast tier, cut retrieval depth where evals allow. Cost: Batch API for non-urgent bulk, premium tiers only for the slice that measurably needs them. And log everything you optimize; without per-step traces you cannot attribute improvements or regressions.

Chapter 6: Governance, safety, and compliance

6.1 The five guardrail layers

Safety questions reward defense in depth, in order:

1. **Input validation and injection screening** before anything reaches the model
2. **System prompt constraints**: role, boundaries, refusal behavior
3. **Tool permission scoping** so the agent cannot act outside its mandate
4. **Output filtering and moderation** before responses reach users or systems
5. **Human review** for the high-stakes residue the earlier layers flag

A single layer is never the answer to "how do you guarantee safe behavior." Prompt instructions are probabilistic; permission scoping and output validation are deterministic, so compliance-critical controls must include the programmatic layers.

The five layers as a request path, each one independent of the model's cooperation from layer 3 outward:

```
def handle(user_input: str, user: User) -> str:
    screened = injection_screen(user_input)            # 1. input validation
    if screened.blocked:
        return refusal(screened.reason)

    response = client.messages.create(
        model=MODEL,
        system=CONSTRAINED_SYSTEM_PROMPT,               # 2. role, boundaries, refusal behavior
        tools=scope_tools_for(user),                    # 3. least-privilege tool set per
        caller
        messages=[{"role": "user", "content": screened.text}],
    )

    verdict = output_filter(response)                   # 4. moderation before delivery
    if verdict.requires_review:
        return queue_for_human(response, verdict)       # 5. human on the flagged residue
    return verdict.text
```

6.2 Human-in-the-loop placement

- A human sits **before irreversible or high-impact actions**: payments, deletions, external communications, contract commitments.
- **Low-confidence and policy-ambiguous outputs route to review via deterministic thresholds set by the system.** A model's self-reported confidence is poorly calibrated and never the trigger.

- Where **regulation demands human accountability**, the human is mandatory regardless of measured accuracy.
- Keep humans **out of high-volume, low-risk paths**; sample there instead. A human approving every routine action destroys the efficiency case.

6.3 Compliance frameworks

Framework	One line	Triggered by
GDPR	EU personal-data regulation: lawful basis, minimization, right to erasure	Processing EU residents' personal data, wherever you sit
HIPAA	US health-information law: safeguards and vendor BAAs	Patient/health data for US covered entities
FedRAMP	US federal cloud authorization program	Cloud services for US federal agencies

Match framework to **data type and jurisdiction**. US health data → HIPAA. EU users → GDPR. Federal agency deployment → FedRAMP. Scenarios can trigger more than one.

6.4 Ethics and failure modes

Bias: evaluate per demographic/language/regional slice; aggregates hide per-slice failures. Fairness: consistent criteria, disparate-outcome testing before launch, monitoring after. Transparency: documented intended use and limitations, explainable decisions, disclosed AI involvement. Failure modes to name on sight: hallucination, prompt injection (malicious instructions smuggled via inputs or retrieved content), data leakage through tool outputs or logs, over-permissioned agents.

Chapter 7: Stakeholders and the solution lifecycle

The domain that surprises engineers. Nine questions, and none of them are about code.

7.1 Lifecycle phases

Discovery → design → handoff → monitoring → iteration. Questions test the order and each phase's outputs. Starting to build before discovery outputs exist is wrong at the professional level, full stop.

7.2 Discovery outputs

- A business problem statement with **measurable success criteria** ("reduce average handling time by a defined target", never "make support better")
- A **constraint inventory**: latency budgets, cost ceilings, compliance regimes, data residency
- A **data inventory and access map**: what exists, where it lives, who owns it, what the agent may touch
- A **stakeholder map with decision owners**, so trade-off calls have a named approver
- A **current-process baseline**, because improvement claims need a before number

7.3 SLAs for probabilistic systems

Translate model behavior into explicit targets before building: latency percentiles, an accuracy floor with an **error budget**, cost per transaction. Set probabilistic expectations early: LLM systems produce a measurable error rate rather than deterministic correctness, so commit to measured rates plus a remediation path instead of promising perfection. Revisit SLAs at every phase boundary.

7.4 Architecture decision records

An ADR is a short, version-controlled document capturing one consequential decision: context, options considered, decision, consequences. One per significant choice (pattern, tier, integration mechanism, guardrail design). On the exam, ADRs are the answer to "how should the team preserve the reasoning behind this architecture."

The ADR format, small enough to actually get written:

```
# ADR-007: Workflow pattern for contract summarization (not agentic)

## Status
Accepted, 2026-08-12

## Context
Legal requires identical five-section briefs with a reviewable processing
trail. Compliance sign-off depends on deterministic, auditable steps.

## Options considered
1. Agentic loop with document tools: flexible, but non-deterministic paths
   fail the auditability requirement
2. Fixed workflow, one model call per section, all inputs/outputs logged
3. Single augmented call: cheapest, but cannot produce per-step trails

## Decision
Option 2. Auditability is the binding constraint and only fixed steps satisfy it.

## Consequences
Per-contract cost is ~5x a single call. New section types require a code
change rather than a prompt change. Revisit if the audit requirement is
relaxed or section count exceeds 10.
```

An SLA for a probabilistic system commits to measured rates and remediation rather than perfection:

Commitment	Target	Measured by	On breach
Latency	p95 under 4s	Per-request traces	Route review, capacity add
Accuracy floor	96% on the held-out eval set, per slice	Weekly eval run	Error budget consumed; change freeze until recovered
Cost	Under $0.04 per transaction	Usage logs	Lever review (caching, routing, batch)
Escalation	100% of flagged outputs reach human review in 4h	Queue metrics	Staffing or threshold adjustment

7.5 Communication

Quantify trade-offs in business terms ("the capable tier adds cost per request but lifts resolution rate" is architect language; raw benchmarks alone are engineer language). Run stakeholder feedback loops at phase boundaries, while corrections are cheap. Handoff means documentation: runbooks, prompt inventories, eval baselines, escalation paths.

Chapter 8: Team enablement

Four questions, mostly one idea: **team-wide behavior belongs in version-controlled, project-level configuration.** A project CLAUDE.md committed to the repository reaches every teammate; user-level config stays personal and never propagates. Shared MCP servers go in project-scoped config with `${ENV_VAR}` references for secrets. Permissions and allowed tools scope to least privilege in shared and CI environments, mirroring Chapter 3's principle. Enablement also covers workflow support: AI-assisted review, debugging assistance, documented conventions.

Decision rule: when a question asks how an entire team reliably gets a behavior, the answer lives in version-controlled project configuration, never in any individual's local setup.

One level above the team sits **managed policy**, the organization-wide layer IT deploys to developer machines (via MDM, Group Policy, or similar), which individual settings cannot exclude:

Platform	Managed CLAUDE.md location
macOS	`/Library/Application Support/ClaudeCode/CLAUDE.md`
Linux / WSL	`/etc/claude-code/CLAUDE.md`
Windows	`C:\Program Files\ClaudeCode\CLAUDE.md`

The division of labor mirrors the guardrail principle from Chapter 6: **managed settings enforce** (blocked tools, denied permissions, sandbox requirements, login restrictions), while **managed CLAUDE.md guides** (coding standards, compliance reminders). An enterprise scenario asking how to *prevent* an action across the organization wants managed settings `permissions.deny`; one asking how to *standardize behavior* wants the managed CLAUDE.md. Instructions shape what the model does; settings are enforced by the client regardless of what the model decides.

Part II: Exam domain notes

Domain 1: Solution Design & Architecture (17%)

Know: the three patterns and their selection rules; the four-part architecture (input, processing, output, feedback loop); decomposition by capability vs data domain; sequential vs parallel orchestration; the three business pillars.

Be able to: pick the simplest sufficient pattern from a scenario; justify (or refuse) multi-agent decomposition; anchor a recommendation to the named business constraint.

Anti-patterns: agentic-by-default; architectures missing the feedback loop; decomposition without independent subtasks.

Domain 2: Models, Prompting & Context Engineering (13%)

Know: the three tiers and their selection logic; routing; zero-shot vs few-shot vs chain-of-thought; system prompt ordering; prompt caching mechanics, payoff conditions, and the three cache-breaking mistakes; edge-of-window attention.

Be able to: default to balanced and move on eval evidence; structure a prompt stable-first for caching; write testable criteria.

Anti-patterns: flagship-everywhere; timestamps early in cached prefixes; vague criteria ("be careful").

Domain 3: Integration (19%)

Know: MCP vs direct API vs agent-to-agent selection tells; capability bloat signs; progressive discovery vs monolithic context; per-user authorization; secret management; invocation attributability.

Be able to: choose the mechanism from reuse and independence requirements; spot and fix bloat by scoping; identify the shared-service-account audit gap.

Anti-patterns: the mega-agent; protocol layers on one-off integrations; credentials in prompts or committed config.

Domain 4: Evaluation, Testing & Optimization (16%)

Know: the metric-to-failure table; eval dataset design rules; grader/judge/human division of labor; A/B discipline; the four-row failure diagnosis table; the optimization levers.

Be able to: diagnose hallucination vs retrieval failure vs tier mismatch vs prompt failure from symptoms; design a held-out, production-shaped eval set; run one clean experiment.

Anti-patterns: tuning prompts on the test set; multi-variable experiments; "feels better" rollouts; aggregate metrics without slices.

Domain 5: Governance, Safety & Risk (14%)

Know: the five guardrail layers in order; HITL placement rules and non-triggers; GDPR/HIPAA/FedRAMP one-liners and triggers; the named failure modes.

Be able to: design defense in depth for a compliance requirement; place humans by risk, sampling elsewhere; match frameworks to data and jurisdiction (including multiple at once).

Anti-patterns: single-layer "guarantees"; prompt instructions as compliance controls; all-or-nothing HITL; the wrong framework for the jurisdiction.

Domain 6: Stakeholder Communication & Lifecycle (14%)

Know: the five phases in order; the five discovery outputs; SLA structure for probabilistic systems; what an ADR captures; handoff artifacts.

Be able to: refuse to design before discovery; convert model behavior into commitments with error budgets; write the ADR answer when asked how to preserve reasoning.

Anti-patterns: skip-discovery; deterministic-correctness promises; handoff without runbooks and baselines.

Domain 7: Developer Productivity & Enablement (7%)

Know: project-level vs user-level configuration and version control; shared MCP config with env-var secrets; least privilege in shared/CI environments.

Be able to: place a team behavior at the correct config level on sight.

Anti-patterns: team standards in personal config; raw secrets in committed files.

Part III: Exam preparation

The preparation ladder

Five layers, in order. Each builds on the previous, and the free layers take you surprisingly far. Anthropic retired its official practice exam when the exams moved to Pearson VUE; the sample questions in the official exam guide are what remains.

Step	Resource	Cost
1. Learn the vocabulary	Anthropic Partner Academy courses, the official exam guide, and the Anthropic Academy courses mapped in Resources	Free
2. Go to the source	docs.anthropic.com: integration, agents, evaluation, guardrails	Free
3. First calibration	30 practice questions with explanations in this repo, also as a timed practice test	Free
4. Full calibration	Six full-length 63-question practice tests and a 500-card flashcard deck on Preporato, built on the exact domain weights; the page has free sample questions to try first	Paid
5. Prove it hands-on	18 auto-graded build-and-submit projects at the architecture level	Paid

Steps 1-3 cost nothing and tell you honestly where you stand. Steps 4-5 are how you close the gap they reveal.

The nine trap answers

Professional-level distractors are plausible and technically true in some context. These patterns mark an option wrong for the scenario at hand:

Trap	Why it fails
Flagship-everywhere	Ignores cost and latency pillars; tier must match difficulty and volume
Agentic-by-default	Autonomy without a reason is a defect; simplest sufficient pattern wins
Prompt-guarantee	Compliance and safety enforcement comes from permissions, validation, layers
Context-stuffing	Retrieval precision and prompt structure are the levers, not volume
Mega-agent	Capability bloat degrades selection and widens the blast radius
Skip-discovery	Architect answers establish metrics, constraints, and access first
All-or-nothing HITL	Placement is risk-based: humans on irreversible/regulated, sampling on routine
Aggregate-metric	Stratified per-slice evaluation is the professional answer
Single-framework	Framework must match data type and jurisdiction, sometimes several at once

For multiple-response items, apply the traps to each option independently. Every selection must survive on its own; one bloated or prompt-guarantee option among your picks costs the whole question, because there is no partial credit.

Exam-day strategy

120 minutes, 63 questions: just under 2 minutes each. The scenario stems are long, so read the **question line first**, then the scenario, so you know what to extract. Identify the domain, find the named business constraint (cost, latency, compliance, reuse), and check each option against the trap table. Flag and move past anything consuming over 3 minutes; final 10 minutes for flagged reviews, changing answers only on certain error.

Worked questions

Twelve fresh questions in the exam's style. (Written for this guide; timed full-length practice is linked in Resources.)

Q1. A legal team needs contracts summarized into a fixed five-section brief, every time, with an auditable processing trail for compliance sign-off. Which pattern?

- A) Agentic loop with document tools, for flexibility across contract types
- B) A fixed workflow of model calls, one per section, with logged inputs and outputs
- C) A single augmented call with the contract in context
- D) Multi-agent decomposition with a contract-type router

Answer: B. Fixed output structure plus auditability is the workflow tell. A and D add autonomy and complexity the requirement forbids rather than needs; C cannot produce the per-step trail compliance wants.

Q2. A support assistant answers 50,000 queries daily. Evals show the fast tier handles 85% of traffic at target accuracy but fails on complex multi-policy questions. The architect's move?

- A) Upgrade all traffic to the top capability tier
- B) Keep the fast tier and add more few-shot examples for the hard cases
- C) Route: fast tier triages and handles routine traffic, escalating the hard slice to a capable tier
- D) Fine-tune the fast tier on the failing queries

Answer: C. Routing captures the cost of the easy 85% and the quality of the hard 15%. A pays flagship prices for solved traffic; B applies a prompting fix to a capability gap; D proposes heavier machinery where routing solves it directly.

Q3. An agent's prompt begins with the current timestamp, then the system prompt, tool definitions, and reference documents. Prompt caching saves almost nothing. Why?

- A) The reference documents exceed the cacheable length
- B) The timestamp at the start changes every request, invalidating the cached prefix from position zero
- C) Tool definitions cannot be cached

- D) The cache TTL is shorter than the request interval

Answer: B. A changing token early in the prefix breaks everything after it. The fix is stable-first, variable-last ordering. C is false, and A and D invent facts the scenario does not give.

Q4. Three product teams want the same internal knowledge-search capability from Claude Code, a desktop assistant, and a customer-facing agent. Integration mechanism?

- A) Each team builds a direct API integration for full control
- B) One MCP server exposing the search capability to all three surfaces
- C) Agent-to-agent delegation to a search agent
- D) A shared prompt template with the search logic described

Answer: B. Build once, expose everywhere across multiple client surfaces is MCP's defining case. A triples maintenance, C adds autonomous-system machinery for a shared tool, D is a prompt where a protocol is required.

Q5. A RAG system's answers are frequently wrong. Traces show the top retrieved chunks are usually off-topic. The team proposes raising retrieval from 5 chunks to 20. Assessment?

- A) Correct: more chunks raise the odds the right one is included
- B) Wrong: retrieval precision is the problem, so more chunks add more noise; fix chunking, embeddings, or query construction, and consider re-ranking
- C) Correct, but only with a larger context window
- D) Wrong: the model tier should be upgraded first

Answer: B. Context-stuffing is the trap; when precision is low, volume amplifies noise and cost. D misdiagnoses (the failure table points at retrieval, since the wrong documents arrive).

Q6. (Select TWO) An underwriting assistant recommends loan approvals. Which two controls are required rather than optional?

- A) Human review before any final approval decision reaches a customer
- B) A system prompt instructing the model to be fair and unbiased
- C) Per-slice evaluation of recommendation quality across demographic groups
- D) The top capability tier for all underwriting traffic
- E) Human review of every internal draft the model produces

Answer: A and C. Irreversible, regulated, high-impact decisions demand HITL placement (A), and fairness in a regulated decision demands stratified evaluation (C). B is a prompt-guarantee, D is flagship-everywhere, E is all-or-nothing HITL applied to a low-risk internal step.

Q7. Two months after launch, an extraction system's judge scores drift upward while customer complaints rise. What most likely broke?

- A) The model was silently upgraded
- B) The LLM judge's rubric no longer matches human quality standards; recalibrate the judge against sampled human review
- C) The eval dataset is too small

- D) Latency increased

Answer: B. Diverging judge scores and human perception is the calibration failure mixed-methodology testing exists to catch: sample, compare, recalibrate. The others do not explain the divergence pattern.

Q8. A pilot's stakeholders are unhappy: the system works but "isn't what we needed." A post-mortem finds no written success criteria, no constraint inventory, and no baseline of the process it replaced. Which phase failed?

- A) Design, for choosing the wrong pattern
- B) Monitoring, for missing quality regressions
- C) Discovery, whose outputs (success criteria, constraints, baseline) were never produced
- D) Handoff, for missing documentation

Answer: C. All three missing artifacts are discovery outputs. The system "working" while missing the need is precisely what skipped discovery produces.

Q9. Compliance asks how the team will preserve the reasoning behind choosing a workflow pattern over an agentic one, so future maintainers understand the constraint. Answer?

- A) A comment block in the orchestration code
- B) An architecture decision record capturing context, options, decision, and consequences, version controlled with the project
- C) A recorded meeting where the decision was discussed
- D) The system prompt, which encodes the behavior

Answer: B. This phrasing is the ADR question, nearly verbatim. Comments (A) capture what, rarely why-not-alternatives; C is unsearchable and unversioned; D encodes behavior rather than reasoning.

Q10. A platform team wants every engineer's Claude Code to enforce the same review conventions and reach the same internal MCP servers. Where does the configuration live?

- A) Each engineer's `~/.claude/CLAUDE.md`, distributed by onboarding doc
- B) Version-controlled project-level configuration: committed CLAUDE.md and project-scoped MCP config with environment-variable secrets
- C) A shared service account whose settings all engineers inherit
- D) A wiki page documenting the conventions

Answer: B. Team-wide behavior lives in committed project config. A depends on every individual complying forever, C is the shared-account authorization anti-pattern from Domain 3, and D is invisible to the tooling.

Q11. An enterprise must guarantee that no developer's Claude Code session can ever call external network tools, across every repository on every machine, and separately wants company coding standards applied everywhere. Which pairing is correct?

- A) Both in a managed CLAUDE.md deployed to all machines
- B) Both in managed settings

- C) The network-tool ban in managed settings (permission deny rules, which the client enforces), the coding standards in the managed CLAUDE.md (behavioral guidance)
- D) The ban in each project's committed settings, the standards in each project's CLAUDE.md

Answer: C. Enforcement and guidance live in different layers: settings are enforced by the client regardless of what the model decides, while CLAUDE.md shapes behavior. A puts a hard requirement in a soft layer, B puts prose standards in an enforcement mechanism that cannot express them, and D covers only repositories that adopt it, missing "every repository on every machine."

Q12. Six months post-launch, the extraction system's weekly eval still scores 97%, but enterprise customers in one region report rising errors. Investigation shows the eval set was built from launch-week traffic and never refreshed, while regional document formats changed. What failed?

- A) The model drifted and should be re-pinned
- B) Eval dataset maintenance: the set no longer mirrors production distribution, so its score stopped measuring reality; refresh it and add the new formats as a scored slice
- C) The accuracy target was set too low
- D) The judge rubric needs recalibration

Answer: B. An eval frozen at launch measures the past. The score stayed high because the test stayed easy, which is exactly why eval design requires refreshing as traffic drifts and scoring per slice. A invents model drift against a pinned version, C tunes a number the measurement no longer supports, and D points at the judge when the dataset is what decayed.

6-week study plan

- **Weeks 1-2: baseline and gap map.** Check that your employer is in the Claude Partner Network (registration needs your company email), complete the Partner Academy architect courses, read the official exam guide, and map your experience against all seven domains honestly. Most candidates' gap is governance or stakeholder work, and this guide's Chapters 6-7 exist for exactly that.
- **Weeks 3-4: the enterprise domains.** Work through docs.anthropic.com on integration and evaluation while practicing the judgment tables here (pattern selection, failure diagnosis, framework matching). If you have never run discovery or written an ADR, do both on a hypothetical engagement end to end. The hands-on projects (paid) cover this ground with automatic rubric grading if you want the builds checked.
- **Weeks 5-6: calibrate.** Start with the 30 practice questions in this repo to see the format, then take full-length, timed, 63-question practice tests. With seven domains, a comfortable average can hide a failing stakeholder or governance score, so read the per-domain breakdown ruthlessly and drill the weak two. Final week: this guide's Part III daily, the cheat sheet the night before.

Expanded week-by-week version: 6-week CCAR-P study plan.

Pre-exam checklist

- [ ] I can choose augmented LLM vs workflow vs agentic from a scenario, defaulting to the simplest sufficient pattern

- [ ] I can name the four parts of a complete architecture, including the feedback loop
- [ ] I can justify or refuse multi-agent decomposition
- [ ] I can anchor recommendations to efficiency, cost, or performance SLA pillars
- [ ] I can pick a model tier from capability, cost, and latency, and know when routing beats one model
- [ ] I know when prompt caching pays off and all three cache-breaking mistakes
- [ ] I can select MCP vs direct API vs agent-to-agent from reuse and independence tells
- [ ] I can spot capability bloat and explain progressive discovery
- [ ] I know why per-user authorization beats shared service accounts
- [ ] I can reason chunk-size trade-offs, know the re-ranking trigger, and fix retrieval before stuffing context
- [ ] I can match metrics to failure modes and design a held-out, production-shaped eval set
- [ ] I know the grader / judge / human division and judge calibration
- [ ] I can run a clean A/B test and diagnose failures from the symptom table
- [ ] I can list the five guardrail layers in order
- [ ] I know HITL placement rules and the non-triggers (sentiment, self-reported confidence)
- [ ] I can distinguish GDPR, HIPAA, and FedRAMP in one line each and match them to scenarios
- [ ] I can name the lifecycle phases in order and all five discovery outputs
- [ ] I can structure an SLA for a probabilistic system with an error budget
- [ ] I know what an ADR captures and when it is the answer
- [ ] I know why team standards live in version-controlled project config
- [ ] I have the nine trap patterns down cold and apply them per-option on multiple-response items
- [ ] I have taken at least two full-length timed practice tests and reviewed every miss

Resources

Official (free):

- Anthropic Partner Academy: the exam guide, prep courses, and registration
- docs.anthropic.com: integration, agents, evaluation, guardrails
- Anthropic Academy courses, mapped to this exam's domains:
  - Building with the Claude API: prompting, tool use, RAG, agents, MCP, production patterns
  - Introduction to Model Context Protocol and MCP: Advanced Topics: the 19% integration domain
  - Claude with Amazon Bedrock and Claude on Google Cloud: enterprise deployment surfaces the solution-design scenarios draw on
  - Claude Code in Action: the developer productivity and enablement domain

Free articles (preporato.com):

- CCAR-P complete guide · Cheat sheet · Domains breakdown · Exam format · How to pass first attempt
- Topic guides: enterprise deployment (Bedrock/Vertex/Foundry) · governance, guardrails & HITL · cost & latency optimization · evaluation strategy · practice questions with explanations

Practice:

- 30 practice questions with explanations in this repo, also as a timed practice test (free)
- The CCAR-P page on Preporato: free sample questions to start, then the full tests below
- (Paid) Six full-length 63-question practice tests and a 500-card flashcard deck built on the exact domain weights
- (Paid) 18 hands-on projects: build-and-submit work at the architecture level graded automatically against a rubric

---

Not affiliated with Anthropic. Exam details reflect the program as of October 2026; the Partner Academy is authoritative. Found an error? Open an issue.

Practice questions

30 questions for CCAR-P, written for this guide in the exam's style and spread across its domains by the official weights. Questions that ask for more than one answer say how many. The answers and explanations follow in the next section.

The same questions run as a timed test with a score for every domain: practice test.

Question 1 *(Solution Design & Architecture)*

A support team wants answers to product questions grounded in a 300-page manual. Questions are single-turn, and the manual is updated monthly. What is the simplest sufficient architecture?

- **A.** An augmented LLM: retrieval over the manual feeding a single model call that cites the passages it used
- **B.** An autonomous agent that browses the manual page by page
- **C.** A multi-agent research system with a planner and three specialists
- **D.** A model fine-tuned on the manual every month

Question 2 *(Solution Design & Architecture)*

Which architecture proposal for an invoice-processing assistant is complete?

- **A.** Extraction with schema validation; posting to the ERP; a dashboard of monthly volume
- **B.** Email intake and OCR; extraction with schema validation; posting to the ERP with an exceptions queue; per-field accuracy evals, user corrections and monitoring fed back into prompt and model decisions
- **C.** Email intake and OCR; extraction with the most capable model; posting to the ERP
- **D.** Email intake; a multi-agent extraction team; posting to the ERP; a launch announcement

Question 3 *(Solution Design & Architecture)*

Two designs meet the accuracy target. Design 1 uses the top tier: p95 latency 6 seconds, $0.09 per request. Design 2 routes between the fast and balanced tiers: p95 latency 2 seconds, $0.02 per request. The client's stated constraint is a 3-second p95 SLA at 2 million requests a month. Which design, and why?

- **A.** Run both in production indefinitely
- **B.** Postpone the decision until a newer model is released
- **C.** Design 1, for quality headroom
- **D.** Design 2, because it meets the latency SLA and the cost pillar at the same accuracy

Question 4 *(Solution Design & Architecture)*

A due-diligence system must analyze each target company's financials, legal filings and press coverage. The three analyses are independent, and each needs different tools and sources. Which design fits?

- **A.** Three separate systems whose reports nobody combines

- **B.** One agent holding all the tools for all three analyses
- **C.** An orchestrator with three scoped subagents running in parallel, followed by a synthesis step
- **D.** One agent doing the three analyses in sequence

Question 5 *(Solution Design & Architecture)*

A global retailer's policy assistant covers 14 regions. Each region's policies differ and are owned by different teams. Answers mix rules from different regions. What is the best design change?

- **A.** Put all 14 regions' policies in every request
- **B.** Partition by data domain: tag documents with their region and filter retrieval by the user's region before semantic search
- **C.** Fourteen agents that negotiate with each other on every question
- **D.** Ask users to mention their region and change nothing else

Question 6 *(Claude Models, Prompting & Context Engineering)*

Which TWO workloads benefit most from prompt caching? (Select TWO)

- **A.** Single requests shorter than the minimum cacheable length
- **B.** Requests whose first line is the current timestamp
- **C.** An agent loop that resends the same 12,000-token system prompt and tool definitions on every turn
- **D.** An endpoint where many users ask questions about the same 50-page policy within minutes of each other
- **E.** A weekly job that sends one unique 3,000-token prompt

Question 7 *(Claude Models, Prompting & Context Engineering)*

An eligibility step must apply six interacting rules. Accuracy is the priority, and the latency budget is generous. Which prompting approach fits?

- **A.** A lower temperature
- **B.** Few-shot examples alone, with no stated rules
- **C.** Have the model reason through the rules step by step (or enable thinking) before deciding, with each rule stated as an explicit criterion, then validate the decision
- **D.** A zero-shot prompt kept as short as possible

Question 8 *(Claude Models, Prompting & Context Engineering)*

An enterprise assistant's system prompt has grown to 60,000 tokens of procedures for 40 business processes, and most requests need one or two of them. Latency and cost are high, and instruction-following has slipped. What is the best restructuring?

- **A.** Build 40 separate assistants and make users choose
- **B.** Move to a model with a larger context window
- **C.** Mark the most important procedures in capitals

- **D.** Keep the system prompt to the core role and rules, and package each procedure as a Skill or retrievable module that loads only when relevant

Question 9 *(Claude Models, Prompting & Context Engineering)*

Downstream systems consume the assistant's JSON, and occasional schema violations cause failed transactions. Which change addresses this at the source, and what still needs checking?

- **A.** Structured outputs with the integration schema, which guarantees schema-valid JSON, plus semantic validation of the values and handling of the refusal and `max_tokens` stop reasons
- **B.** A stronger instruction to follow the schema
- **C.** A retry loop and nothing else
- **D.** A regex that repairs broken JSON

Question 10 *(Integration)*

A procurement agent must get quotes from a supplier's own AI agent, which runs on the supplier's systems with its own tools, policies and reasoning. Which integration mechanism fits?

- **A.** Agent-to-agent delegation over a defined protocol, with explicit request and response contracts, authentication, and failure isolation
- **B.** A direct API call to Claude with the supplier's price list pasted into the prompt
- **C.** An MCP server that gives your agent direct access to the supplier's database
- **D.** A shared system prompt that both companies edit

Question 11 *(Integration)*

One internal app sends support tickets to Claude for tagging. One team owns it, and no reuse is planned. A platform engineer proposes building an MCP server first. What is the right assessment?

- **A.** Fine-tune a tagging model instead
- **B.** Build the MCP server, since every integration should use MCP
- **C.** Use a direct API integration now; MCP adds a protocol layer and maintenance without a reuse need, so revisit it if other surfaces need the capability
- **D.** Use agent-to-agent delegation

Question 12 *(Integration)*

Which TWO observations are signs of capability bloat in a production agent? (Select TWO)

- **A.** Two tools have overlapping descriptions, and both plausibly handle 'find a customer'
- **B.** Each subagent's tools are scoped to its responsibility
- **C.** The reporting agent uses read-only credentials
- **D.** It exposes 45 tools, and invocation logs show only 9 ever used
- **E.** Tool descriptions state when not to use each tool

Question 13 *(Integration)*

An agent connects to six MCP servers exposing 300 tools, and every request front-loads all of their definitions. Tool selection is poor and costs are high. What is the best fix?

- **A.** Add more examples to the system prompt
- **B.** Shorten every tool description to one line
- **C.** Move to a model with a larger context window
- **D.** Progressive discovery: load tool definitions on demand (for example with tool search and deferred loading), and give each agent only the servers its responsibility needs

Question 14 *(Integration)*

A company assistant answers questions over the document store using a service account that can read every folder, including HR and legal. A contractor asked about salaries and got an answer. What is the root fix?

- **A.** Remove HR documents from the index
- **B.** Run retrieval and tools with the calling user's identity and permissions, and attribute every access to that user in the logs
- **C.** Add a system-prompt rule: 'Never discuss salaries'
- **D.** Filter out responses that contain the word 'salary'

Question 15 *(Integration)*

Engineers search an internal knowledge base with exact error codes such as 'E-4031.' Semantic search returns conceptually similar documents about other errors. What is the best fix?

- **A.** A larger embedding model
- **B.** Hybrid search: keyword matching for exact identifiers combined with semantic search, then re-ranking
- **C.** Retrieve more chunks per query
- **D.** Fine-tune the generation model on error codes

Question 16 *(Evaluation, Testing & Optimization)*

A contract-extraction service has three complaints: wrong values, slow responses at peak hours, and a rising monthly bill. Which THREE metrics should the monitoring plan track? (Select THREE)

- **A.** Field-level accuracy against a labeled dataset
- **B.** Average response length
- **C.** p95 and p99 latency
- **D.** Number of prompt revisions per month
- **E.** Cost per document and tokens per task

Question 17 *(Evaluation, Testing & Optimization)*

A team tuned its prompt against the same 200 examples it uses to report accuracy. Testing shows 98%, and production shows 84%. What went wrong, and what is the fix?

- **A.** The eval set leaked into prompt development and inflated the score; tune on a separate development set and report on a held-out set that mirrors production
- **B.** The model is too small; upgrade the tier
- **C.** Production traffic is low quality; filter it
- **D.** 200 examples is too few; add 20 more of the same kind

Question 18 *(Evaluation, Testing & Optimization)*

A team wants to ship a new prompt, new retrieval settings and a tier change together, because 'it felt better in 15 spot checks.' What is the correct approach?

- **A.** Run 15 more spot checks
- **B.** Ask the stakeholders which version they prefer
- **C.** Test one variable at a time with a success metric and sample size set in advance, then ship each change only on a significant result
- **D.** Ship all three, since the spot checks were positive

Question 19 *(Evaluation, Testing & Optimization)*

Traces show the correct policy passages are retrieved, yet answers still state the wrong refund window. Where is the failure, and what is the first fix?

- **A.** Raise the number of retrieved passages
- **B.** The index is stale: rebuild it
- **C.** Retrieval failure: re-chunk and re-embed the corpus
- **D.** Generation is poorly grounded: tighten the instructions to answer only from the retrieved passages and cite them, and evaluate faithfulness

Question 20 *(Evaluation, Testing & Optimization)*

Answers about leave policies leave out exceptions. Traces show the retrieved chunks contain the rule sentence, while the exception sits in the next paragraph, which isn't retrieved. What is the best fix?

- **A.** Retrieve 30 chunks instead of 5
- **B.** Move to a more capable model
- **C.** Tell the model to remember that policies have exceptions
- **D.** Re-chunk along the document's structure (whole sections) with some overlap, so rules and their exceptions stay together

Question 21 *(Governance, Safety & Risk Management)*

An agent reads inbound customer emails and can issue refunds. Which TWO controls are deterministic, as opposed to prompt-based? (Select TWO)

- **A.** An output filter that blocks any reply containing a full account number
- **B.** Few-shot examples of the agent declining suspicious requests
- **C.** A system-prompt rule: 'Never issue large refunds'
- **D.** A persona instruction: 'You are a careful agent'

- **E.** Tool permissions that cap a refund at $100 per call and send larger amounts for human approval

Question 22 *(Governance, Safety & Risk Management)*

A moderation pipeline handles 500,000 low-risk comments a day and occasionally receives legal threats. Which human-in-the-loop design fits?

- **A.** Have a human review every comment
- **B.** Automate routine decisions with sampled QA, and route legal threats and cases below system-set thresholds to human reviewers
- **C.** Remove humans entirely
- **D.** Let the model escalate whenever its self-reported confidence is low

Question 23 *(Governance, Safety & Risk Management)*

A US telehealth company with patients in the US and Germany plans to use Claude to summarize clinical notes. Which TWO frameworks apply? (Select TWO)

- **A.** FedRAMP
- **B.** HIPAA, for US patients' health information
- **C.** GDPR, for the personal data of patients in the EU
- **D.** None, since Claude is a third-party service
- **E.** Only HIPAA, since the company is based in the US

Question 24 *(Governance, Safety & Risk Management)*

Your observability stack stores full prompts and responses, including customer personal data, in a log platform that 300 engineers can read. What is the risk, and what is the fix?

- **A.** Data leakage through logs: redact personal data before logging, restrict access to the people who need it, and set a retention limit
- **B.** Ask the model to leave personal data out of its responses
- **C.** Encrypt the stored logs and leave access as it is
- **D.** No risk, because the logs are internal

Question 25 *(Stakeholder Communication & Lifecycle Management)*

A vice president asks you to 'make customer support better with AI.' What should you produce first?

- **A.** A comparison of model benchmark scores
- **B.** A working prototype to show next week
- **C.** Discovery outputs: a measurable success criterion, a constraint inventory, a data and access map, a stakeholder map with decision owners, and a baseline of the current process
- **D.** A signed vendor contract

Question 26 *(Stakeholder Communication & Lifecycle Management)*

Stakeholders ask for a guarantee that the assistant 'never gives a wrong answer.' What is the appropriate response?

- **A.** Commit only to 99.9% uptime
- **B.** Promise 100% accuracy to secure approval
- **C.** Refuse to set any accuracy target
- **D.** Commit to measured targets, such as at least 96% accuracy per slice on a held-out set with an error budget, escalation paths and a remediation plan, and explain that LLM outputs have a measurable error rate

Question 27 *(Stakeholder Communication & Lifecycle Management)*

Which statement best communicates a model-tier recommendation to business stakeholders?

- **A.** 'We chose the newest model available.'
- **B.** 'The top tier scores higher on industry benchmarks.'
- **C.** 'The balanced tier resolves 91% of tickets at $0.02 each, and the top tier resolves 93% at $0.08. At a million tickets a month, the extra two points cost about $60,000 a month, so we recommend the balanced tier with escalation for complex tickets.'
- **D.** 'The balanced tier is fine.'

Question 28 *(Stakeholder Communication & Lifecycle Management)*

Which TWO artifacts belong in the handoff to the team that will operate the system? (Select TWO)

- **A.** The architect's personal working notes
- **B.** The kickoff slide deck
- **C.** A raw export of the project chat channel
- **D.** Runbooks covering escalation paths and known failure modes
- **E.** Eval baselines and an inventory of prompts and their versions

Question 29 *(Developer Productivity & Operational Enablement)*

Claude Code runs in your CI pipeline, and the team agrees it must never run git push or curl anywhere the repository is used. Where should this restriction live?

- **A.** Each developer's `~/.claude/settings.json`
- **B.** Deny rules in the committed project settings, such as `Bash(git push *)` and `Bash(curl *)`, which the client enforces
- **C.** A wiki page
- **D.** A `CLAUDE.md` instruction

Question 30 *(Developer Productivity & Operational Enablement)*

A platform team built a code-review subagent and a release-notes skill that every repository in the company should use, in the same version. What is the best way to distribute them?

- **A.** Package them as a plugin (or deploy them through managed settings), so every developer gets the same versioned definitions
- **B.** Post the prompt text in a chat channel for developers to copy
- **C.** Ask each developer to copy the files into `~/.claude/`

- **D.** Paste both into every repository's `CLAUDE.md`

Answers and explanations

Question 1: A. One well-scoped, grounded call meets the requirement, so retrieval plus a single call is the simplest sufficient pattern (A). C and B add autonomy the task doesn't need, with the cost and latency to match. Monthly fine-tuning (D) is heavy, slower to update than re-indexing, and gives no citations.

Question 2: B. A complete architecture has input, processing, output and a feedback loop that turns evals, user corrections and monitoring into design changes (B). C has no feedback loop. A has no input stage, and a volume dashboard measures activity without feeding quality back. D adds complexity and still has no feedback loop.

Question 3: D. When both options meet accuracy, the stated business constraint decides, and only Design 2 meets the 3-second SLA, at about $40,000 a month against $180,000 (D). C breaks the SLA. A doubles cost and complexity for no reason. B ignores a decision the data already supports.

Question 4: C. Independent subtasks with different tool needs are what justifies decomposition: each subagent gets a narrow toolset and its own context, and the orchestrator merges the results (C). B is capability bloat. D gives up parallelism and mixes three contexts. A skips the synthesis the client actually needs.

Question 5: B. The data has a clean partition, so filtering on region metadata keeps other regions' rules out of the context (B). C adds coordination machinery to a retrieval problem. A is context stuffing. D leaves the mixed retrieval in place.

Question 6: C and D. Caching pays off when a long, stable prefix repeats within the cache lifetime (C, D). A weekly unique prompt (E) never repeats in time. A timestamp at the start (B) breaks the prefix on every request. Prompts under the minimum length (A) are never cached.

Question 7: C. Multi-step logic benefits from explicit reasoning before the answer, and the generous latency budget covers its cost (C). Zero-shot (D) suits simple, unambiguous tasks. Examples (B) teach format and edge cases, and six interacting rules still need to be stated. Temperature (A) adds no reasoning.

Question 8: D. Loading procedures on demand keeps the context small and focused, which improves attention, cost and latency together (D). Emphasis (C) doesn't shrink the prompt. A bigger window (B) holds more of the same dilution. A moves the routing problem onto users.

Question 9: A. Constrained decoding removes schema violations at the source, and semantic checks plus stop-reason handling cover what it doesn't guarantee (A). Retries (C) and instructions (B) reduce failures without eliminating them. Regex repair (D) guesses at intent and can corrupt data quietly.

Question 10: A. Two autonomous systems with separate owners, tools and policies call for delegation between agents, with contracts and isolation so either side can fail independently (A). C bypasses the supplier's agent and policies and assumes access you won't get. B works only if you already have current prices, which is what you're asking for. D merges two organizations' control into one prompt.

Question 11: C. MCP pays off when one capability serves many clients or surfaces, and one app with one owner is the direct-API case (C). B adds maintenance for a benefit that doesn't exist yet. D adds autonomous-system machinery to a single call. A skips a simpler option that works.

Question 12: A and D. Unused tools that ship with every request (D) and overlapping tools that blur selection (A) are classic signs of bloat. Scoped toolsets (B), read-only credentials for a read-only job (C) and boundary statements in descriptions (E) are the cure.

Question 13: D. Loading tools as the agent narrows in cuts tokens and improves selection accuracy, and scoping servers per responsibility removes tools that never belonged (D). A bigger window (C) holds the same noise. One-line descriptions (B) make selection worse. Examples (A) add tokens without reducing the tool count.

Question 14: B. The shared over-privileged account erased per-user authorization and made every access unattributable, so the fix is acting as the caller (B). C is a probabilistic patch, A breaks legitimate HR use, and D is easy to get around and blocks legitimate answers.

Question 15: B. Exact identifiers such as error codes and SKUs need keyword matching, which hybrid search adds alongside semantic retrieval (B). Better embeddings (A) still treat codes as fuzzy meaning. More chunks (C) add noise. The generation model (D) can't fix documents that were never retrieved.

Question 16: A, C, E. Each complaint maps to a metric: wrong values to accuracy (A), slowness to tail latency (C), and the bill to cost per unit of work (E). Prompt revision counts (D) and response length (B) measure activity, and neither shows whether the complaints are fixed.

Question 17: A. Tuning against the test set overfits to it, so the score stops predicting production (A). C blames the data the system has to handle. B changes the model without fixing the measurement. D keeps the same leak with a slightly bigger set.

Question 18: C. A clean experiment changes one variable and decides on a metric chosen in advance (C). With three changes at once (D), a win or a loss can't be attributed to anything. Preference (B) and more spot checks (A) are still the feels-better method.

Question 19: D. When the right documents arrive and the answer is still wrong, the failure sits in generation and grounding (D). C and B fix retrieval, which the traces show is working. A adds noise to a context that already contains the answer.

Question 20: D. Chunks smaller than the unit of meaning split rules from their qualifiers, so structure-aware chunking fixes the retrieval unit (D). More chunks (A) might catch the exception along with plenty of noise. No model (B) can use a paragraph it never receives, and an instruction (C) can't supply the missing text.

Question 21: A and E. Permission limits (E) and output filtering (A) are enforced in code, whatever the model decides. C, D and B shape behavior probabilistically, and a cleverly written email can talk the model past them.

Question 22: B. Humans go where the risk is high, and sampling covers the routine volume (B). Reviewing everything (A) destroys the efficiency case. C leaves legal threats unhandled. Self-reported confidence (D) is poorly calibrated, so the thresholds have to be set by the system.

Question 23: B and C. The framework follows the data type and the jurisdiction: US health data brings HIPAA (B), and processing EU residents' personal data brings GDPR wherever the company is based (C). FedRAMP (A) covers cloud services for US federal agencies. E ignores the German patients, and D confuses using a vendor with being exempt.

Question 24: A. Logs and traces are a leakage channel, so the fix minimizes what's stored and who can see it (A). Internal access (D) still exposes customer data to hundreds of people. Encryption with unchanged access (C) protects the storage while every reader still sees the data. B leaves the personal data in the prompts, and the model can't be relied on to keep it out of responses.

Question 25: C. Professional engagements start with discovery, because every later design choice depends on what success means and which constraints apply (C). A prototype (B) built without discovery answers a question nobody has defined. Benchmarks (A) and contracts (D) come after the requirements are known.

Question 26: D. Probabilistic systems get SLAs built on measured rates, error budgets and remediation (D). B sets up a broken promise. C leaves stakeholders nothing to hold the system to. Uptime (A) says nothing about answer quality.

Question 27: C. Architect-level communication puts the trade-off in business terms: outcome, cost, and the recommendation that follows from them (C). B is engineering language with no business meaning. D gives no basis for the decision, and A offers novelty as a reason.

Question 28: D and E. Operators need to know how to respond to problems (D) and what normal looks like, along with what is deployed (E). Personal notes (A), kickoff slides (B) and raw chat (C) aren't operational documentation.

Question 29: B. Permission rules in version-controlled project settings apply everywhere the repository runs, CI included, and the client enforces them (B). User settings (A) don't reach CI or other developers. `CLAUDE.md` (D) is guidance the model weighs. A wiki (C) is invisible to the tooling.

Question 30: A. Plugins bundle skills and subagents for distribution, and managed settings can push them to the whole organization, which keeps one maintained version (A). Copy-and-paste distribution (B, C) drifts as soon as the originals change. `CLAUDE.md` (D) holds always-on instructions and can't define a subagent or an on-demand skill.

Keep going

- **Full-length practice.** The CCAR-P page on Preporato has free sample questions and six full-length timed practice tests (paid) in the exam's format, with a score for every domain.
- **Updates and corrections.** The latest edition, the practice test and the other three guides are at github.com/preporato/claude-certification-guide.